

Business Logic Flow Vulnerabilities — Interview Prep Notes

1. Logical Description

- Flaw in **intended workflow/rules**, not a technical/injection bug
- App behaves "as designed" — design itself permits abuse
- No malformed input required — valid requests, wrong sequence/values/timing
- Exploits assumptions: **sequence, quantity, state, ownership, timing**
- Root cause: business rules enforced client-side or trusted from client, not re-validated server-side

2. OWASP Category & CWE

- OWASP Top 10 2021: **A04:2021 – Insecure Design** (primary bucket)
- Overlaps A01 (Broken Access Control) when logic bypass = privilege/state escalation
- OWASP Testing Guide: **WSTG-BUSL-01 to 09** (dedicated Business Logic Testing category)
- CWE-840 – Business Logic Errors (parent)
- CWE-841 – Improper Enforcement of Behavioral Workflow
- CWE-372 – Incomplete Internal State Distinction
- CWE-362 – Race Condition (TOCTOU)
- CWE-682 – Incorrect Calculation
- CWE-799 – Improper Control of Interaction Frequency
- CWE-807 – Reliance on Untrusted Inputs in Security Decision

3. Common Variants

- Price/quantity manipulation (negative qty, decimals, currency swap)
- Coupon/discount abuse — stacking, reuse, expired-code replay
- Workflow/step skipping (direct API call bypasses UI-enforced step)
- Race conditions (TOCTOU) — double spend/redeem/withdraw
- State machine bypass (e.g., cancel-after-ship, approve without review)
- Excessive trust in client-side validation/hidden fields
- Referral/loyalty/reward program abuse
- Free trial / account creation abuse (infinite resets)
- Insufficient anti-automation on sensitive bulk actions
- OTP/password-reset flow ordering flaws

4. Attack Surface / Entry Points

- Checkout/cart APIs (price, qty, coupon params)
- Multi-step wizards (step N called directly, skipping N-1)
- Payment gateway callback/webhook endpoints
- State transition endpoints (approve/reject/cancel/refund)
- Referral, loyalty, rewards, wallet endpoints
- Signup / password-reset / OTP-verify flow
- Any concurrent-access endpoint: balance, booking, voting, inventory
- Client-orchestrated multi-request workflows (mobile app, SPA)

5. Technical Testing Workflow

Step	Verify	Manual	Burp Suite	OWASP ZAP	Nuclei	Payload/Test
Map workflow	Full sequence of legit steps/states	Walk UI, note every request	Proxy history + Logger	HUD + History tab	N/A (recon only)	—
Step-skipping	Can step N be called w/o N-1?	Replay step N request directly, drop prior steps	Repeater — replay out of order	Manual Request Editor	No (logic-specific)	Call <code>/confirm</code> w/o <code>/cart</code> or <code>/payment</code> first
Parameter tampering	Server trusts client-sent business values?	Modify price/qty/total in request	Repeater/Intruder on price/qty/total field	Fuzzer on same params	Custom template (regex on response diff)	<code>price=0.01</code> , <code>qty=-1</code> , <code>total=1</code>
State/role reuse	Object allowed in unintended state?	Move order to invalid state via API	Repeater, chain sequential calls	Same	No	Cancel after "shipped", refund after "delivered"
Race condition	Server enforces atomicity?	Fire duplicate requests near-simultaneously	Turbo Intruder (single-packet attack), or Repeater "Send group in parallel"	ZAP doesn't natively race well — use external script	No	20-50 parallel redeem/withdraw requests, single coupon/balance
Coupon/reward reuse	Is code single-use enforced server-side?	Reapply same code after use	Repeater replay	Fuzzer replay	Custom template if endpoint fixed	Reuse same coupon ID across sessions
Rate/quantity abuse	Business limit enforced server-side?	Exceed documented limit (max qty, max redemptions)	Intruder — sweep quantities	Fuzzer	No	qty = 99999, redemptions > allowed cap
Currency/unit confusion	Mixed currency/locale exploited?	Submit mismatched currency code vs price	Repeater	Manual	No	<code>currency=JPY</code> w/ USD-scale price

- Note in interview: Nuclei is **weak fit** — business logic needs stateful, sequential, context-aware testing; templates can't model multi-step workflows well. Mention this as a known tooling gap.

6. Evidence to Collect

- Full request/response chain (ordered) proving sequence deviation
- Before/after state proof (balance, order total, inventory count)
- Timestamps for race condition (Turbo Intruder attack log, response codes/timing)
- Screenshot/video of unintended end-state (e.g., "Order confirmed" at \$0)
- Server logs showing accepted out-of-sequence/duplicate request

7. Good PoC

- Ordered repro steps (numbered), each with request/response
- Clear statement of expected vs actual business outcome
- Quantified impact ("Purchased ₹5000 item for ₹1", "Redeemed coupon 40x = ₹8000 loss")
- Video/screenshot of final state (order, wallet, dashboard)
- Reproducibility note (works every time / race-dependent — include success rate)

8. Attack Chains

- Logic flaw + IDOR → manipulate/view another user's order/cart
- Race condition + no locking → fund duplication / wallet drain
- Step-skip + broken auth → bypass verification/KYC step entirely
- Price manipulation + no server recompute → systemic fraud at scale
- Coupon abuse + no rate limiting → mass exploitation via automation
- Workflow bypass + weak session mgmt → account takeover via reset-flow skip

9. Common False Positives

- Client-side-only bypass with **no actual server impact**
- Behavior expected in staging/test/sandbox environments
- Discount stacking that is business-approved (documented risk acceptance)
- Rate-limit trigger misread as logic flaw
- UI restriction absent server-side but genuinely low-risk by design (accepted)

10. Prerequisites for Exploitability

- Server trusts client for business-critical values/state
- No server-side state machine validation
- No transaction locking / idempotency keys
- Direct API access possible (no strict workflow gating via UI-only tokens)
- Insufficient step-sequence enforcement (no server-tracked session workflow state)

11. Attacker Achievements

- Financial fraud (free/underpriced goods, fund duplication)
- Privilege/state escalation via flow bypass
- Resource/service abuse (infinite trials, mass registrations)
- Inventory exhaustion / denial of business
- Compliance/regulatory violation (KYC/AML bypass)
- Account takeover via reset/verification flow abuse

12. CIA Impact (by variant)

Variant	C	I	A	Example CVSS Vector
Race condition (fund/coupon dup)	None	High	Low	AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:L
Price/qty manipulation	None	High	None	AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:N
Workflow/step-skip (bypass verification)	Low–High*	High	None	AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:H/A:N
Coupon/reward reuse	None	Medium	Low	AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:M/A:L (custom, not CVSS-native)
Trial/resource abuse	None	Low	Medium	AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:L/A:M

*depends whether bypassed step gates sensitive data exposure

13. Typical CVSS Severity

- Range: **Medium–Critical**, highly context-dependent (no clean CWE→CVSS mapping since impact is business, not technical)
- Fund duplication / race condition on payments: typically **High (7.5–8.1)**
- Price manipulation on checkout: **High (7.5+)**
- Caveat for interview: **CVSS base score often understates real severity** — supplement with business impact rating (financial exposure, scale of automation feasibility)

14. Business Impact (Financial App Context)

- Direct monetary loss (fraud, duplicated funds, underpriced transactions)
- Regulatory exposure — PCI-DSS, RBI guidelines (India), AML/KYC violations
- Chargeback and reconciliation costs
- Audit failures / regulatory penalties
- Reputational damage, customer trust erosion
- Scalable abuse risk — bot-driven mass exploitation of single logic flaw

15. Indicators of Exploitation

- Duplicate order/transaction IDs in logs
- Spike in specific coupon code usage
- Wallet/balance reconciliation mismatches
- Out-of-sequence API calls in access logs vs expected funnel
- Burst of identical requests within milliseconds (race attempt signature)
- Analytics funnel showing users skipping expected steps

16. Root Cause (Code-Level)

- Business rules validated only in UI/client, not re-checked server-side
- Server trusts client-submitted price/qty/state
- No atomic transaction / row locking → race window
- No state machine enforcement (any state → any state allowed)

```
// Vulnerable: client-trusted value
total = request.total
processPayment(total)

// Vulnerable: race condition, no lock
balance = getBalance(user)
if balance >= amount:
    # window here - concurrent request also passes check
    debit(user, amount)
```

17. Remediation

- Recompute all business-critical values server-side (never trust client price/qty/total)
- Enforce state machine transitions server-side — whitelist valid state→state moves
- Atomic transactions: DB-level locking (`SELECT FOR UPDATE`) or optimistic concurrency (version column)
- Idempotency keys for sensitive operations (payment, redemption)
- Server-authoritative workflow/session tracking — not hidden form fields
- Rate limiting + anti-automation on sensitive endpoints

```
// Fixed
BEGIN TRANSACTION
SELECT balance FROM wallet WHERE user_id=? FOR UPDATE
if balance >= amount:
    debit(user, amount)
COMMIT
```

18. Compensating Controls

- WAF/API gateway rules for abnormal request sequences
- Rate limiting on high-risk endpoints
- Manual review threshold for high-value transactions
- Fraud scoring / anomaly detection (velocity checks)
- Scheduled reconciliation jobs (balance/order audit)
- Alerting on mismatch thresholds

19. If Seen Repeatedly — Indicates

- Threat modeling absent at design phase (no abuse-case analysis)
- No business logic test cases in SDLC / QA regression
- Over-reliance on client-side controls org-wide
- Missing security requirements gathering during user story creation
- Weak "assume breach" culture — dev trusts happy-path only

20. Upstream Improvements

- Mandatory abuse-case/threat modeling at design stage (STRIDE applied to workflows)
- Server-side validation checklist in code review (business-critical fields)
- Shared concurrency-safe library (locking/idempotency pattern) — same pattern as your SAST/shared-wrapper approach for IDOR
- Business logic test cases as first-class QA regression suite (not just DAST)
- Security requirements baked into user stories/acceptance criteria
- Framework-level idempotency key support as default pattern

21. Sample Triage Response

"Confirmed — business logic flaw allows [step-skip/race condition/price manipulation] via [endpoint]. Verified server does not [re-validate state/lock transaction/recompute value], resulting in [quantified impact, e.g., fund duplication of ₹X]. Reproducible [N/N] attempts. Severity: High — direct financial impact, scalable via automation. Recommend immediate rate-limiting as interim control while root-cause fix (server-side recalculation/atomic transaction) is implemented."

22. Sample Developer Remediation Note

"Endpoint `/api/wallet/redeem` does not lock the balance row during redemption, permitting concurrent requests to pass the balance check before either debit commits (TOCTOU). Fix: wrap balance-check + debit in a single DB transaction using `SELECT FOR UPDATE` (or optimistic locking via version column). Additionally add idempotency key per redemption request to reject duplicate submissions. Retest with Turbo Intruder parallel-request PoC post-fix."

23. Real-World Example

- **Starbucks gift card race condition** (Egor Homakov/Sakurity, 2015) — parallel balance-transfer requests exploited missing transaction locking to duplicate gift card funds; widely cited as the canonical race-condition business logic bug
 - Multiple HackerOne-disclosed **referral/coupon abuse** reports (e.g., Shopify, Uber referral fraud cases) — logic allowed unlimited referral credit via account cycling, no server-side uniqueness/velocity check
-